

TP: Les Piles et les Files

Définition 1

On peut déclarer une file d'attente en utilisant la méthode **deque()** de la bibliothèque **collections** comme illustré dans l'exemple suivant.

```
1 from collections import deque
2 # importation à partir de la bibliothèque collections
3
4 # création d'un deque
5 f1= deque([1,2,3,4,5])
6 f2= deque("PSI") # renvoie la file ['P','S','I']
7
8 # suppression à droite
9 f1.pop() # [1,2,3,4]
10
11 # suppression à gauche
12 f1.popleft() # [2,3,4]
13
14 # insérer à droite la valeur 42
15 f1.append(42) # [2,3,4,42]
16
17 # insertion à gauche
18 f2.appendleft('M') # ['M','P','S','I']
```

Exercice 1

Écrire toutes les fonctions primitives pour manipuler une Pile.

Exercice 2: Expression bien parenthésée

On vous donne une expression sous forme de chaîne de caractères avec des parenthèses. Le but est de créer un programme qui vérifie que cette expression est bien parenthésée. Seules les parenthèses suivantes sont prises en considération: "{, [], ()"

Exemple

En entrée: **parenthese**("a+(b+[c*d])") →: *True*

En entrée: **parenthese**("a+[b+[c*d])") → *False*

Pour le résoudre à l'aide d'une pile :

1. On parcourt l'expression de gauche à droite
2. On ne s'intéresse qu'aux parenthèses.
3. Dès que l'on rencontre une parenthèse ouvrante "{", on empile sa partie fermante "}" sur la pile.
4. Dès que l'on rencontre une parenthèse fermante, trois cas se présentent :
 - Si la pile est vide, dans ce cas l'expression n'est pas bien parenthésée.
 - Si la pile est non-vide, dans ce cas on la dépile si le sommet de la pile et la parenthèse rencontrée sont égales.
 - On renvoie *False* sinon
5. A la fin si la pile est vide l'expression est bien parenthésée, sinon elle ne l'est pas.

Implémenter la fonction **parenthese(exp)**.

Exercice 3: Évaluation d'une expression

L'écriture classique d'une expression algébrique nécessite l'utilisation de parenthèses pour regrouper certaines opérations. La notation polonaise inversée présente l'avantage de rendre le parenthésage inutile. Le principe est que les opérateurs (+, -, *, /) sont placés après leurs opérandes (nombres).

Exemple :

- $2+3$ s'écrit $2\ 3\ +$
- $2+3*4$ s'écrit $2\ 3\ 4\ *\ +$
- $(2+3)*4$ s'écrit $2\ 3\ +\ 4\ *\$
- $(2+3)*(4+5)$ s'écrit $2\ 3\ +\ 4\ 5\ +\ *\$

Algorithme à suivre : Il s'agit de définir une fonction qui prend en argument une liste représentant la notation polonaise d'une expression, et retourne un nombre, ou False.

- On parcourt la liste en testant chaque élément :
 - Si c'est un nombre, on l'empile ;
 - si c'est un opérateur, on dépile et lit 2 fois, on effectue l'opération, et on empile le résultat ;
 - sinon on sort car l'expression est incorrecte.
- Dans le cas où on a pu terminer la liste : il ne devrait rester qu'un nombre dans la pile, qui est le résultat ; sinon l'expression est incorrecte.

Implémenter la fonction **polonaise(exp)**.

Exercice 4: Conversion d'une notation polonaise en une expression parenthésée

Cette fois-ci on souhaite convertir une expression en notation polonaise inversée en une expression arithmétique parenthésée sous forme d'une chaîne de caractères comme donnée en exemple:

Exemple :

- $2\ 3\ +$ donne $\rightarrow (2 + 3)$
- $2\ 3\ 4\ *\ +$ donne $\rightarrow (2 + (3 * 4))$
- $7\ 3\ 2\ -\ *\ 15\ 8\ +\ /\$ donne $\rightarrow ((7*(3-2))/(15+8))$

Algorithme à suivre : Le même principe de l'exercice précédent sauf qu'ici au lieu d'évaluer les deux valeurs dépilées, on les concatène avec l'opérateur en question puis on empile le résultat sur la pile.

Implémenter la fonction **polonaise_vers_parenthese(exp)**.

Exercice 5: ****Tours de Hanoï

On considère le célèbre problème des **tours de Hanoï** :

- On dispose de trois tours : A, B et C.
- On a n disques de tailles différentes, empilés sur la tour A du plus grand en bas au plus petit en haut.
- L'objectif est de transférer tous les disques de la tour A vers la tour C en respectant les règles suivantes :
 1. On ne peut déplacer qu'un seul disque à la fois.
 2. On ne peut jamais placer un disque plus grand sur un disque plus petit.

1. Résoudre le problème à la main si $n = 2$ puis $n=3$.

2. Écrire une fonction récursive **hanoi(n, source, intermediaire, destination)** en Python qui affiche la liste des déplacements nécessaires pour transférer n disques de la tour **source** vers la tour **destination** en utilisant la tour **intermediaire**.

3. Bonus : Compter le nombre total de déplacements effectués pour n disques.

